Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики
и радиоэлектроники»

Специальность «Программная инженерия»

Кафедра инженерной психологии и эргономики

Учебная дисциплина «Компьютерные системы и сети»

ОТЧЕТ

по лабораторной работе №3
«P2PChat»

Подготовила: Плехова Е.В.

Проверила: Болтак С.В.

Минск 2026

Задание: Необходимо разработать программу (консольную или графическую) для обмена текстовыми сообщениями, работающую в локальной сети в одноранговом режиме.

Каждый участник обмена сообщениями (узел) идентифицируется IP-адресом и произвольным именем, которое задается пользователем (через параметр командной строки, конфигурационный файл или любым другим способом). Уникальность имен не требуется.

Каждый узел с помощью UDP формирует список активных узлов (IP-адреса и имена): после запуска узел отправляет широковещательный пакет, содержащий свое имя, для уведомления других узлов в сети о своем подключении к сети; другие узлы, получившие такой пакет, устанавливают с отправителем TCP-соединение для обмена сообщениями и передают по нему свое имя для идентификации в чате.

В любой момент к чату может присоединиться новый клиент.

Обмен сообщениями ведется с помощью TCP в логически общем пространстве: каждый узел поддерживает по одному TCP соединению с каждым другим узлом и отправляет свои сообщения всем узлам в сети. Отключение узла должно корректно обрабатываться другими узлами.

Пользовательский интерфейс программы должен позволять вводить с клавиатуры и отправлять сообщения, а также просматривать историю событий с момента последнего запуска программы.

История должна включать следующие сообщения в хронологическом порядке с отметками времени:

– входящие сообщения от других узлов (с указанием имени и IP-адреса отправителя);

– собственные отправленные сообщения;

– обнаружение нового узла;

– отключение работающего узла.

Для обмена сообщениями рекомендуется разработать свой собственный формат сообщения, позволяющий передавать сообщения разных типов и упрощающих передачу сообщений в потоковом режиме, используемом в TCP.

При создании сокетов на прослушивание входящих широковещательных UDP-пакетов и TCP-соединений нужно указывать IP-адрес, переданный при запуске программы (параметр командной строки, конфигурационный файл и т.п.). Далее запустить несколько экземпляров программы, передавая разные IP-адреса: 127.0.0.1, 127.0.0.2 и т.д. Все эти адреса будут ассоциированы с вашим компьютером, однако логически программы будут разными независимыми клиентами.

Дополнительное задание.

Реализовать передачу имеющейся истории событий узлу при подключении.

После обнаружения узлов новый узел запрашивает у любого имеющегося историю событий. По TCP-соединению, установленному между этими узлами, происходит передача полной истории событий, известной узлу.

После приема история должна быть отображена соответствующим образом в пользовательском интерфейсе.

Решение.

Лабораторная работы выполнена в среде разработки Visual Studio 2022 на языке C#. Программа реализует одноранговую (peer-to-peer) сеть для обмена текстовыми сообщениями. Каждый узел одновременно является и сервером (принимает входящие соединения), и клиентом (подключается к другим узлам). Приложение использует асинхронное программирование (async/await) для выполнения нескольких задач одновременно.

Принцип работы программы:

1. Обнаружение узлов (UDP Broadcast).

При запуске каждый узел отправляет UDP широковещательный пакет (broadcast) на порт, содержащий свое имя и TCP-порт.

Алгоритм:

1. Узел запускается → создает UDP сокет, привязанный к своему IP.
2. Отправляет broadcast на адрес сети (например, 10.0.0.255:9999).
3. Другие узлы получают broadcast и узнают о новом узле.
4. Узел-получатель инициирует TCP-соединение с отправителем.

UDP:

- Поддерживает широковещательную рассылку;
- Не требует установки соединения;
- Потеря пакета не критична (узел отправит повторный broadcast).

2. Установка TCP-соединения.

После обнаружения узла устанавливается TCP-соединение для надежного обмена данными.

TCP:

- Гарантирует доставку сообщений;
- Сохраняет порядок пакетов;
- Обеспечивает надежное соединение.

3. Формат сообщений.

Каждое сообщение имеет бинарный формат с JSON-содержимым:

Тип | Длина сообщения | JSON-данные

Типы сообщений задаются в перечислении «enum MessageType»:

```
public enum MessageType : byte
{
    TextMessage = 1, // Текстовое сообщение чата
    NameExchange = 2, // Обмен именами пользователей
    UserConnected = 3, // Уведомление о подключении узла
    UserDisconnected = 4, // Уведомление об отключении узла
    HistoryRequest = 5, // Запрос истории сообщений
    HistoryResponse = 6, // Ответ с историей сообщений
    Ping = 7, // Проверка активности узла
    Pong = 8 // Ответ на Ping
}
```

4. Обмен сообщениями

Отправка сообщения:

1. Пользователь вводит текст в консоли
2. Программа создает объект «ChatMessage» с типом «TextMessage»
3. Сообщение сериализуется в формат [тип][длина][JSON]
4. Сообщение отправляется всем подключенным узлам через TCP
5. Сообщение сохраняется в локальную историю и файл

Получение сообщения:

1. Узел получает TCP-пакет
2. Из буфера извлекается сообщение по префиксу [тип][длина]
3. JSON десериализуется в объект «ChatMessage»
4. Сообщение выводится в консоль с именем отправителя
5. Сообщение сохраняется в историю и файл

5. Передача истории

При подключении нового узла происходит синхронизация истории:

Алгоритм передачи истории:

1. Существующий узел отправляет HistoryResponse со всей своей историей
2. Новый узел получает историю и добавляет недостающие сообщения
3. Новый узел отправляет HistoryRequest для запроса истории
4. Существующий узел отвечает HistoryResponse (если есть новые сообщения)
5. История сортируется по времени для сохранения хронологии

6. Поддержание соединений (Ping/Pong)

Для обнаружения отключившихся узлов используется механизм ping-pong:

Каждые 60 секунд узел отправляет ping всем другим узлам. Если узел не отвечает на 3 ping подряд (180 секунд), он считается stale и удаляется из списка активных узлов. Это позволяет корректно обрабатывать отключения без явного уведомления.

7. Хранение истории

В памяти история хранится в «List<HistoryEntry>». Максимальный размер: 500 записей (для предотвращения переполнения).

В файле:

- Имя файла: «chat_history_Имя_ДатаВремя.txt»
- Формат: `[2026-03-23 21:21:05] [Alice (10.0.0.1)]: Hello`
- Записываются все типы событий: сообщения, подключения, отключения.

10. Обработка ошибок и отключений

Корректное завершение (Ctrl+C или `/quit`):

1. Отправляется broadcast «UserDisconnected»
2. Закрываются все TCP-соединения
3. Останавливаются TCP и UDP слушатели

4. В историю добавляется запись об отключении
5. История сохраняется в файл

Обнаружение отключения другого узла:

1. TCP-соединение разрывается (Exception при чтении)
2. Узел удаляется из списка активных
3. В консоль выводится сообщение: Bob (10.0.0.2:9002) left the chat
4. В историю добавляется запись об отключении

Запустим через командную строку 4 узла чата с разных loopback-адресов, разных tcp портов и одного udp порта (пример запуска узла представлен на рисунке 1)

```
D:\Programist\ksis\P2PChat\bin\Debug\net8.0>p2pchat
=== P2P Chat System ===

Available IP addresses on this computer:
  1. 192.168.10.124
  2. 192.168.56.1
  3. 10.0.0.4
  4. 10.0.0.3
  5. 10.0.0.2
  6. 10.0.0.1

Enter your IP address (127.0.0.1): 10.0.0.1
Enter your TCP port (unique for this node, 9001): 9001
Enter UDP port for broadcast discovery (9999): 9999
Enter your name: Liza
```

Рисунок 1 – Запуск узла

Когда в чате будет только первый узел, отправим сообщение, чтобы проверить корректность передачи истории новым узлам. (рис. 2)

```
Командная строка - p2pchat
=== P2P Chat Node: Liza ===
Local IP: 10.0.0.1
TCP Port: 9001
UDP Port: 9999
History file: D:\Programist\ksis\P2PChat\bin\Debug\net8.0\chat_history_Liza_20260324_182812.txt
==================================================
[OK] TCP server listening on 10.0.0.1:9001
[OK] UDP listener bound to 10.0.0.1:9999

Chat started!
Commands:
  /quit - exit
  /nodes - show active nodes
  /history - show message history
  /connect <ip> <port> - manually connect to a node
--------------------------------------------------
Hello, I am Liza
[You]: Hello, I am Liza
```

Рисунок 2

У последнего подключенного узла проверим историю чата, и увидим, что она полная, т е при подключении нового узла к чату ему передается полная история чата. (рис. 3)

```
Выбрать Администратор: Командная строка - p2pchat
=== P2P Chat Node: Pen ===
Local IP: 10.0.0.4
TCP Port: 9004
UDP Port: 9999
History file: D:\Programist\ksis\P2PChat\bin\Debug\net8.0\chat_history_Pen_20260324_183500.txt
==================================================
[OK] TCP server listening on 10.0.0.4:9004
[OK] UDP listener bound to 10.0.0.4:9999

Chat started!
Commands:
  /quit - exit
  /nodes - show active nodes
  /history - show message history
  /connect <ip> <port> - manually connect to a node
--------------------------------------------------

*** Lilo (10.0.0.3:9003) joined the chat ***


*** Dan (10.0.0.2:9002) joined the chat ***


*** Liza (10.0.0.1:9001) joined the chat ***

/history

=== Message History ===
[18:28:12] Chat session started by Liza (10.0.0.1)
[18:28:36] [Liza (10.0.0.1)]: Hello, I am Liza
[18:32:01] Chat session started by Dan (10.0.0.2)
[18:32:16] [Dan (10.0.0.2)]: qwerty
[18:34:13] Chat session started by Lilo (10.0.0.3)
[18:35:00] Chat session started by Pen (10.0.0.4)
```

Рисунок 3

Теперь проверим отключение узлов. Для этого третий узел сети покинет чат (рис. 4). При отключении узла появляется соответствующее сообщение в чате, также добавляются записи в историю чата и в файл (рис 5).

```
--------------------------------------------------
Hello, I am Liza
[You]: Hello, I am Liza

*** Dan (10.0.0.2:9002) joined the chat ***


[Dan]: qwerty


*** Lilo (10.0.0.3:9003) joined the chat ***


*** Pen (10.0.0.4:9004) joined the chat ***


[Lilo]: the weather is good


*** Dan (10.0.0.2:9002) left the chat ***


*** Lilo (10.0.0.3:9003) left the chat ***

/history

=== Message History ===
[18:28:12] Chat session started by Liza (10.0.0.1)
[18:28:36] [Liza (10.0.0.1)]: Hello, I am Liza
[18:32:01] Chat session started by Dan (10.0.0.2)
[18:32:16] [Dan (10.0.0.2)]: qwerty
[18:34:13] Chat session started by Lilo (10.0.0.3)
[18:35:00] Chat session started by Pen (10.0.0.4)
[18:39:13] [Lilo (10.0.0.3)]: the weather is good
[18:39:18] Dan (10.0.0.2) left the chat
[18:40:11] Lilo (10.0.0.3) left the chat
```

Рисунок 4

D:\Programist\ksis\P2PChat\bin\Debug\net8.0\chat_history_Dan_20260324_183201.txt - Notepad++

Файл Правка Поиск Вид Кодировки Синтаксисы Опции Инструменты Макросы Запуск Плагины Вкладки ?

logbook.log | chat_history_Bob_20260323_212246.txt | chat_history_Liza_20260323_212149.txt | chat_history_Dan_20260324_183201.txt

```
1  [SESSION START] Chat session started by Dan at 10.0.0.2:9002
2  [2026-03-24 18:32:01] Chat session started by Dan (10.0.0.2)
3  [2026-03-24 18:28:12] Chat session started by Liza (10.0.0.1)
4  [2026-03-24 18:28:36] [Liza (10.0.0.1)]: Hello, I am Liza
5  [2026-03-24 18:32:16] [Dan (10.0.0.2)]: qwerty
6  [2026-03-24 18:34:13] Chat session started by Lilo (10.0.0.3)
7  [2026-03-24 18:35:00] Chat session started by Pen (10.0.0.4)
8  [2026-03-24 18:39:13] [Lilo (10.0.0.3)]: the weather is good
9  [2026-03-24 18:39:18] Dan (10.0.0.2) left the chat
10 [SESSION END] Chat session ended at 2026-03-24 18:39:18
11 ============================================================
12 [2026-03-24 18:39:18] Liza (10.0.0.1) left the chat
13 [2026-03-24 18:39:18] Pen (10.0.0.4) left the chat
14 [2026-03-24 18:39:18] Lilo (10.0.0.3) left the chat
15
```

Рисунок 5

Для того, чтобы узнать список участников чата, нужно использовать команду /nodes (рис 6)

```
/nodes

=== Active Nodes ===
Liza at 10.0.0.1:9001
Pen at 10.0.0.4:9004
Total: 2 nodes
```

Рисунок 6

Теперь проанализируем трафик в Вайшарк. При отключении и подключении узла к чату будет отправляться udp broadcast сообщение (рис 7-9)

```
▸ Frame 7: Packet, 173 bytes on wire (1384 bits), 173 bytes captured (1384 bits) on interface \Device\NPF_
▸ Null/Loopback
▸ Internet Protocol Version 4, Src: 10.0.0.2 (10.0.0.2), Dst: 10.0.0.255 (10.0.0.255)
▾ User Datagram Protocol, Src Port: 51799 (51799), Dst Port: distinct (9999)
    Source Port: 51799 (51799)
    Destination Port: distinct (9999)
    Length: 149
    Checksum: 0xd58a [unverified]
    [Checksum Status: Unverified]
    [Stream index: 1]
    [Stream Packet Number: 1]
  ▸ [Timestamps]
    UDP payload (141 bytes)
▾ Data (141 bytes)
    Data […]: 04008a7b2254797065223a342c22436f6e74656e74223a22466c6164222c2253656e6465724e616d65223a22466
    [Length: 141]
```

Рисунок 7

```
02 00 00 00 45 00 00 a9  de 91 00 00 80 11 00 00   ····E··· ********
0a 00 00 02 0a 00 00 ff  ca 57 27 0f 00 95 d5 8a   ******** ·W'·····
04 00 8a 7b 22 54 79 70  65 22 3a 34 2c 22 43 6f   ···{"Typ e":4,"Co
6e 74 65 6e 74 22 3a 22  46 6c 61 64 22 2c 22 53   ntent":" Flad","S
65 6e 64 65 72 4e 61 6d  65 22 3a 22 46 6c 61 64   enderNam e":"Flad
22 2c 22 53 65 6e 64 65  72 49 70 22 3a 22 31 30   ","Sende rIp":"10
2e 30 2e 30 2e 32 22 2c  22 53 65 6e 64 65 72 54   .0.0.2", "SenderT
63 70 50 6f 72 74 22 3a  39 30 30 32 2c 22 54 69   cpPort": 9002,"Ti
6d 65 73 74 61 6d 70 22  3a 22 32 30 32 36 2d 30   mestamp" :"2026-0
33 2d 32 34 54 31 38 3a  35 31 3a 30 33 2e 35 34   3-24T18: 51:03.54
30 31 37 30 36 2b 30 33  3a 30 30 22 7d            01706+03 :00"}
```

Рисунок 8

2	7.163273	WIN-BGSMG2ERFTG.loc…	mdns.mcast.net	MDNS	129	Standard query response 0x0000 AAAA
7	21.282465	10.0.0.2	10.0.0.255	UDP	173	51799 → distinct(9999) Len=141
9	23.560218	WIN-BGSMG2ERFTG.loc…	192.168.10.255	BROWSER	233	Host Announcement WIN-BGSMG2ERFTG,
10	23.563897	192.168.56.1	192.168.56.255	BROWSER	233	Host Announcement WIN-BGSMG2ERFTG,
13	37.172903	WIN-BGSMG2ERFTG.loc…	mdns.mcast.net	MDNS	129	Standard query response 0x0000 AAAA
21	74.553154	10.0.0.3	10.0.0.255	UDP	171	61738 → distinct(9999) Len=139

Рисунок 9

Обмен сообщениями в чате происходит по tcp-протоколу (рис 10). В Вайшарк ip источника и назначения отображаются одинаковыми из-за использования loopback-интерфейса, но на самом деле узлы находятся на разных ip-адресах (выше мы их запускали с разных ip. Верные ip указываются в самом чате и при записи истории в файл).

4	3.805491	10.0.0.3	10.0.0.3	TCP	187	49842 → 9003 [PSH, ACK] Seq=1 Ack=1 Win=1255 Len=143
5	3.805547	10.0.0.3	10.0.0.3	TCP	44	9003 → 49842 [ACK] Seq=1 Ack=144 Win=8402 Len=0
7	12.466429	10.0.0.3	10.0.0.3	TCP	186	9003 → 49842 [PSH, ACK] Seq=1 Ack=144 Win=8402 Len=142
8	12.466515	10.0.0.3	10.0.0.3	TCP	44	49842 → 9003 [ACK] Seq=144 Ack=143 Win=1254 Len=0
15	32.889765	10.0.0.3	10.0.0.3	TCP	186	9003 → 49842 [PSH, ACK] Seq=143 Ack=144 Win=8402 Len=142
16	32.889829	10.0.0.3	10.0.0.3	TCP	44	49842 → 9003 [ACK] Seq=144 Ack=285 Win=1253 Len=0
21	56.595329	10.0.0.3	10.0.0.3	TCP	183	9003 → 49842 [PSH, ACK] Seq=285 Ack=144 Win=8402 Len=139
22	56.595395	10.0.0.3	10.0.0.3	TCP	44	49842 → 9003 [ACK] Seq=144 Ack=424 Win=1253 Len=0
23	56.597135	10.0.0.3	10.0.0.3	TCP	183	49842 → 9003 [PSH, ACK] Seq=144 Ack=424 Win=1253 Len=139
24	56.597221	10.0.0.3	10.0.0.3	TCP	44	9003 → 49842 [ACK] Seq=424 Ack=283 Win=8402 Len=0

Рисунок 10 – Обмен сообщениями

4	13.925993	10.0.0.2	10.0.0.2	TCP	56	57106 → dynamid(9002) [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
5	13.926044	10.0.0.2	10.0.0.2	TCP	56	dynamid(9002) → 57106 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
6	13.926083	10.0.0.2	10.0.0.2	TCP	44	57106 → dynamid(9002) [ACK] Seq=1 Ack=1 Win=2161152 Len=0

Рисунок 11 – Установка соединения

Wireshark · Пакет 9 · Adapter for loopback traffic capture

```
▸ Frame 9: Packet, 185 bytes on wire (1480 bits), 185 bytes captured (148
▸ Null/Loopback
▸ Internet Protocol Version 4, Src: 10.0.0.2 (10.0.0.2), Dst: 10.0.0.2 (1
▾ Transmission Control Protocol, Src Port: 57106 (57106), Dst Port: dynam
    Source Port: 57106 (57106)
    Destination Port: dynamid (9002)
    [Stream index: 0]
    [Stream Packet Number: 6]
  ▸ [Conversation completeness: Incomplete, DATA (15)]
    [TCP Segment Len: 141]
    Sequence Number: 1    (relative sequence number)
    Sequence Number (raw): 441741956
    [Next Sequence Number: 142    (relative sequence number)]
    Acknowledgment Number: 141    (relative ack number)
    Acknowledgment number (raw): 3602903877
    0101 .... = Header Length: 20 bytes (5)
  ▸ Flags: 0x018 (PSH, ACK)
    Window: 8442
    [Calculated window size: 2161152]
    [Window size scaling factor: 256]
    Checksum: 0xc851 [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
  ▸ [Timestamps]
  ▸ [SEQ/ACK analysis]
    [Client Contiguous Streams: 1]
    [Server Contiguous Streams: 1]
    TCP payload (141 bytes)
▾ Data (141 bytes)
    Data […]: 02008a7b2254797065223a322c22436f6e74656e74223a224c697a6122
```

Рисунок 12 – Содержимое пакета

```
02 00 00 00 45 00 00 b5  f0 77 40 00 80 06 00 00   ····E···  ·w@·····
0a 00 00 02 0a 00 00 02  df 12 23 2a 1a 54 72 84   ********  ··#*·Tr·
d6 bf f3 45 50 18 20 fa  c8 51 00 00 02 00 8a 7b   ···EP··· ·Q·····{
22 54 79 70 65 22 3a 32  2c 22 43 6f 6e 74 65 6e   "Type":2 ,"Conten
74 22 3a 22 4c 69 7a 61  22 2c 22 53 65 6e 64 65   t":"Liza ","Sende
72 4e 61 6d 65 22 3a 22  4c 69 7a 61 22 2c 22 53   rName":" Liza","S
65 6e 64 65 72 49 70 22  3a 22 31 30 2e 30 2e 30   enderIp" :"10.0.0
2e 31 22 2c 22 53 65 6e  64 65 72 54 63 70 50 6f   .1","Sen derTcpPo
72 74 22 3a 39 30 30 31  2c 22 54 69 6d 65 73 74   rt":9001 ,"Timest
61 6d 70 22 3a 22 32 30  32 36 2d 30 33 2d 32 34   amp":"20 26-03-24
54 31 39 3a 30 32 3a 31  37 2e 31 39 32 36 34 37   T19:02:1 7.192647
35 2b 30 33 3a 30 30 22  7d                        5+03:00" }
```

Рисунок 13 – Содержимое пакета

Вывод: В ходе выполнения лабораторной работы было разработано консольное приложение для обмена текстовыми сообщениями в локальной сети, работающее в одноранговом (P2P) режиме. Программа реализует следующий функционал: автоматическое обнаружение узлов через UDP broadcast, надежную передачу сообщений по TCP, синхронизацию истории чата при подключении новых участников, а также сохранение истории в файл с подробными отметками времени.

Архитектура приложения построена таким образом, что каждый узел одновременно выступает в роли сервера (принимает входящие TCP-соединения через TcpListener) и клиента (инициирует соединения к другим узлам через TcpClient). Для обнаружения узлов используется UDP-протокол: при запуске каждый узел отправляет широковещательный пакет, содержащий свое имя, IP-адрес и TCP-порт. Другие узлы, получившие этот пакет, устанавливают TCP-соединение с отправителем. Обмен сообщениями ведется по протоколу TCP, что гарантирует надежную доставку и сохранение порядка сообщений. Для передачи данных разработан собственный формат сообщений, состоящий из заголовка (тип сообщения – 1 байт, длина JSON-данных – 2 байта) и тела в формате JSON, которое содержит текст сообщения, имя отправителя, его IP-адрес, TCP-порт и временную метку.

Благодаря асинхронному программированию (async/await) программа одновременно обрабатывает ввод пользователя, принимает новые TCP-соединения, слушает UDP broadcast, отправляет ping и принимает сообщения от всех подключенных узлов. При корректном завершении работы (Ctrl+C или /quit) узел отправляет broadcast об отключении, закрывает все соединения и сохраняет историю в файл.